Abstraction

Data Abstraction

- Data abstraction means **showing only essential features** of an object and **hiding complex internal details**.

- You interact with ***what*** it does, not ***how*** it does it.

- Abstraction makes code:
- Easier to use,
- Easier to maintain,
- Less error-prone.

- A major way to implement abstraction in Python is using **Abstract Base Classes (ABCs)** from the **abc module**.

Abstraction Example

We drive the car, not its engine internals.

Simple Controls Outside

Complex Logic Inside

Abstraction Example

- You use:
  - Steering wheel → turn left/right
  - Pedals → accelerate/brake
  - Buttons → AC, music, lights

- You **don't need to know**:
  - How the engine burns fuel or controls electric motors
  - How the gearbox shifts
  - How the braking system builds hydraulic pressure

You get a **simple interface** (wheel + pedals + buttons), while the **complex machinery is hidden**.

Abstract Base Class (ABC)

An Abstract Base Class (ABC):

- Is a class that **cannot be instantiated** directly.
- Serves as a **blueprint** for other classes.
- Defines a **common interface** (methods/properties) that **subclasses must implement.**

Python

```python
from abc import ABC, abstractmethod
```

- Inherit from **ABC**.
- Use **@abstractmethod** to mark abstract methods.

Abstract Base Class (ABC) Example

Python	Output

`from abc import ABC, abstractmethod`
 	
`class Greet(ABC):`
`  @abstractmethod`
`  def say_hello(self):`
`  pass  # Abstract method`
 	
`class English(Greet):`
`  def say_hello(self):`
`  return "Hello!"`
 	
`g = English()`
`print(g.say_hello())`
	Hello!

Types of Abstraction

There are **three types** of **abstraction**.

- **Data abstraction**: Focuses on **what data is stored and what operations are allowed**, while hiding how the data is actually implemented.
- **Control abstraction**: Focuses on **hiding the flow of control**, meaning the user doesn't need to know *how* tasks are executed internally.
- **Procedural Abstraction**: Focuses on **hiding the implementation of a procedure/function**.

Components of Abstraction

Abstraction in Python (using ABCs) mainly involves:

- **Abstract methods**
- **Concrete methods**
- **Abstract properties**
- **Instantiation rules for abstract classes**

Abstract Method

An **abstract method** is a **method declaration without an implementation** in an abstract class. It forces subclasses to provide their own implementation.

Python

```python
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def honk(self):
        pass  # Abstract method, no implementation here
```

Key points:

- *honk()* has **no body** (just pass).
- Any concrete subclass of Animal **must implement** *honk()*, otherwise it **cannot** be instantiated.

Concrete Method

A **concrete method** is a **normal method with full implementation** inside an abstract class. Subclasses can inherit it as-is, promoting code reuse.

Python

```python
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def honk(self):
        pass  # Abstract method, no implementation here

    def steer_left(self):
        return "Left"

class Car(Vehicle):
    def honk(self):
        return "Honk!!!"
```

Key points:

- **Car** must implement **honk**().
- **Car** inherits **steer_left**() and can use it without redefining.

Abstract Properties

- **Abstract properties** are like abstract methods, but for properties.

- They:
- Use both **@property** and **@abstractmethod**.

- Force **subclasses** to **implement** the property.

Explanation:
- **area** in **Shape** is an abstract property:
- Combination of **@property** + **@abstractmethod**.

- **Square** implements **area**, so:
- **Square** becomes a concrete class,
- An object of **Square** can be created,
- **square.area** returns **area of square**

Abstract properties enforce that subclasses provide a specific property.

Python

```python
from abc import ABC, abstractmethod

class Shape(ABC):
    @property
    @abstractmethod
    def area(self):
        pass  # Abstract property

class Square(Shape):
    def __init__(self, side):
        self._side = side

    @property
    def area(self):
       return self._side ** 2  # Concrete
implementation
```

Abstract Class Instantiation Rules

- You cannot instantiate an abstract class that has **unimplemented abstract methods/properties**.
- Attempting to do so raises a **TypeError**.

Python

```python
from abc import ABC, abstractmethod

class Vehicle(ABC):
    @abstractmethod
    def honk(self):
        pass  # Abstract method, no implementation here

Vehicle = Vehicle()  # TypeError
```

Why?

- **`Vehicle`** is abstract because **`honk()`** is abstract.
- Since **`Vehicle`** itself doesn't define how **`honk()`** works, Python prevents you from creating an incomplete object.

WATCH

Level up your coding with each episode in this focused Python series.

Next Video!

Polymorphism